

SERIAL_SERVO Axis Modes (Kogna)

Axis Input/Output modes that drive Feetech STS3215 smart servos as ordinary KFLOP/Kogna axes. Up to **8 servos daisy chained on a single wire** (IO0, or EX_IO_13 via ServoBusEXIO when IO0/IO1 are needed as encoder 0); each servo's bus ID (0–7) is its axis Input/Output Channel number. Assign IDs once with `SerialServoSetID.c`; see `STS3215_Servos.md` for the bus architecture and wiring.

The modes

Mode	Value	Role
SERIAL_SERVO_INPUT_MODE	Input 5	Position is read from the servo's own encoder instead of an encoder counter or ADC
SERIAL_SERVO_MODE	Output 9	Commanded position is written to the servo. Analogous to STEP_DIR_MODE, except a position is sent rather than a step rate — the servo closes its own loop
CL_SERIAL_SERVO_MODE	Output 10	Same, plus the KFLOP servo loop runs and its Output is added as a correction — analogous to CL_STEP_DIR_MODE

Input Channel and Output Channel are the servo's ID on the bus (0–7).

Configuration

```
ServoBusEXIO = 0;           // bus on IO0 (default), 1 = EX_IO_13

ch0->InputMode  = SERIAL_SERVO_INPUT_MODE;
ch0->InputChan0 = 0;         // servo ID 0
ch0->InputGain0 = 1.0;       // axis units per servo count (4096 counts/rev)

ch0->OutputMode  = SERIAL_SERVO_MODE;
ch0->OutputChan0 = 0;        // same servo
ch0->OutputGain  = 1.0;      // servo counts per axis unit

ch0->MaxFollowingError = 400; // servo counts - protects against a jam
ch0->MaxOutput         = 500; // REQUIRED nonzero for CL mode (see below)
```

No FPGA setup is needed. The DSP protocol engine scans all 16 axes every servo interrupt; whenever any axis claims a serial servo it takes ownership of the bus pin(s), defines them as outputs, and starts running bus cycles. Whenever no transmission or response is pending — between transactions, and while nothing claims the bus at all — the engine PARKS the line: the UART is disabled and the pin actively driven high as a GPIO output, so the single-wire bus never floats (no pull-up resistors are needed, or fitted). A User C program may still drive the bus hardware directly, but must first re-enable the UART by writing `FPGAW(SERVO_BUS_ADD)`.

Torque must still be enabled in the servo — a mechanical decision the DSP deliberately does not make for you:

```
// through the aux mailbox (see SerialServoAxisInit.c for the helper)
ServoWrite(id, STS_REG_TORQUE, 1);
```

Globals sent to every servo each cycle (changeable any time):

```
ServoSpeed = 3400;    // servo speed limit, counts/sec (max published)
ServoAccel = 0;       // 0 = no limit (measured identical to max 254).
                     // NEVER small nonzero - 10 DOUBLES the tracking lag
```

Update rates

Each bus cycle = one SYNC_WRITE broadcasting every actively commanded position + one servo's telemetry read, round robin:

- **Position commands: ~1 kHz** regardless of servo count (72-byte broadcast for 8 servos $\approx 720\ \mu\text{s}$) — matching the servo's ~1 kHz internal update.
- **Telemetry per servo: ~1 kHz / N claimed servos** (~120 Hz with 8, ~500 Hz with 2). With the servo's ~80 ms lag and ~1.6 Hz useful loop bandwidth, even 120 Hz feedback is far more than the loop can use.
- ServoCycleTicks inserts extra 90 μs interrupts between cycles if you want to quiet the bus; 0 (default) runs cycles back to back.

Position is therefore a staircase at the telemetry rate. If you use CL_SERIAL_SERVO_MODE with a D term, low-pass filter the loop (a 2nd-order ~40 Hz filter in one of the axis IIR slots works well).

Position handling

- **Multi-turn works.** The servo reports 0–4095 per revolution; the DSP accumulates with wraparound detection (safe as long as the shaft moves less than half a revolution between that servo's telemetry reads — even at 8 servos that allows ~36 rev/s, ten times the servo's top speed).
- **No homing is needed.** The accumulator seeds from the servo's absolute reading at the first valid telemetry.
- Position is not updated until that first reading, and EnableAxisDest **refuses to enable** a serial servo output axis before then — preventing a full-speed slew toward a stale destination. EnableAxis(ch) enables at the measured position, so enabling never causes motion.

Homing with axis offsets

The goal written to the servo is **absolute in the servo's own count frame**:

```
servo goal = axis command * OutputGain + OutputOffset
```

while the axis position is read back as

```
Position = servo counts * InputGain0 + InputOffset0
```

Because the goal is absolute (unlike Step/Dir, where steps are relative and offsets cancel out), the output transform must be the **inverse** of the input transform: $\text{OutputGain} = 1/\text{InputGain0}$ and $\text{OutputOffset} = -\text{InputOffset0}/\text{InputGain0}$. If they are inconsistent the servo jumps by the mismatch at enable — a large mismatch can overload-fault the servo (power cycle to recover), and in CL_SERIAL_SERVO_MODE the loop fights the displaced servo and parks off position.

Recipe to make a chosen pose read zero (typical gains of 1):

1. With the axis disabled, manually move the joint to the desired zero pose.
2. Read the servo's raw position $P0$ (`ServoPosition[id]`, or `ch->Position` with offsets still zero).
3. Set `InputOffset0 = -P0` and `OutputOffset = +P0` — **negatives of each other**, not equal.
4. Position now reads 0 at the pose and enabling causes no motion.

Requires this release's firmware — older firmware ignored `OutputOffset` in the serial servo output modes, so only zero offsets behaved.

Alternative: keep the axis transforms at unity and store the offset in the servo itself, which shifts its reported and goal positions together — Dynamixel Homing Offset (register 20, range $\pm 1,044,479$ counts) via `Dx1Config.c`, or STS3215 Position Correction (register 31, only ± 2047 counts $\approx \pm$ half turn) via `SerialServoConfig.c`.

Choosing velocity and acceleration

Two limits are easy to trip over:

- **Short moves are acceleration limited:** a move of distance d peaks at $\sqrt{A \cdot d}$, so using velocity V requires $A \geq V^2/d$.
- **The servo has a hard speed ceiling:** ~ 3550 counts/s (7.4 V C001), ~ 3080 counts/s (12 V C018), less under load. Commanding more just grows following error. A C018 does well at $V=2500$, $A=25000$.

Getting the following error near zero

Measured (12 V C018): tracking lag ≈ 80 ms, almost entirely servo-internal dead time. Sweeping the servo's own P gain (reg 0x15) $32 \rightarrow 128$ barely moved it, and `ServoAccel/ServoSpeed/cycle` pacing don't touch it. Dead time is immune to every feedback gain — the servo's and KFLOP's alike — and caps useful loop bandwidth at about $1/(6 \cdot \text{lag}) \approx 2$ Hz, so raising P/D produces oscillation, not tracking.

Feed forward is the fix. In `CL_SERIAL_SERVO_MODE` the loop `Output` is added to the commanded position, and `Output` includes $V_{el} \cdot FF_{Ve1} + Accel \cdot FF_{Accel}$ — so **FF_{Ve1} is a pure command lead in seconds:**

<code>FFVe1</code>	$\approx \text{lag}$	(~ 0.08)	cancels velocity proportional error
<code>FFAccel</code>	$\approx \text{lag} \cdot \text{lag}$	(~ 0.0064)	cancels acceleration induced error

Feed forward sits outside the feedback path and costs no stability margin. `FFAccel` cancels exactly the "raising acceleration makes the error worse" term. Measure the lag rather than guessing — `SerialServoTuneLag.c` jogs at constant velocity (steady-state error / velocity is the lag), then sets both terms. Measure with FF zeroed and PID zeroed (open loop) — the plant lag is what feed forward must match.

With `FFVe1` applied, command and measured position track within ~ 10 – 20 counts through a move; the residual at direction reversals is gear backlash plus the servo's 1-count deadband (regs 0x1A/0x1B), which no tuning removes.

MaxOutput must be nonzero in CL mode. `Output` (the position correction in counts, feed forward included) is clamped to $\pm \text{MaxOutput}$. Left at 0, closed loop silently does nothing. Set it above $FF_{Ve1} \cdot \text{MaxVelocity}$.

Command delay preview (CmdDelay / CmdLead) — the exact lag compensator

Feed forward is a Taylor approximation; **preview is exact for dead time**. Since trajectories are predetermined, the machine can define its official commanded reference as the trajectory delayed by a global worst-case

amount, and command each lagging axis from a tap *ahead* of that reference by its own measured lag — so every axis physically arrives ON the common reference.

```
CmdDelay = 889;           // global, in 90us servo samples (~80 ms)
chan[axis]->CmdLead = 889; // this axis is commanded 80 ms ahead
                          // steppers etc. leave CmdLead = 0
```

What each stream feeds:

- **DelayedDest** (reference = Dest delayed CmdDelay): the Servo() PID error, FF velocity/acceleration derivation, following-error checks, and the Step Response screen's **Command** trace. So each axis is still evaluated *independently* against where it should be right now, exactly as before, and V/A/J error evaluation is unchanged in meaning.
- **LeadDest** (= delayed CmdDelay - CmdLead): the output stages — the serial servo position command, step generator targets, microstep phase. The CL Output correction and FF still add onto it, cleaning up the residual the fixed lead can't know (load variation, the ~5-15 ms gain-dependent lag).

With CmdDelay = 0 (default) everything equals Dest — behavior is identical to before, bit for bit.

Tuning: measure the axis lag (SerialServoTuneLag.c), set CmdLead to it, and the velocity-proportional following error collapses on the Step screen — "on track at constant speed". CmdLead is clamped to CmdDelay, so to lead more, raise both.

The servo's lag is velocity dependent (hand fit $\approx 27\text{ ms} + \sim 84\text{ }\mu\text{s}$ per count/s; measure the real curve with SerialServoLagVsVel.c — constant velocity segments only), so a FIXED lead is exact at one speed and wrong elsewhere: position runs ahead of the reference while accelerating and behind while decelerating.

Do NOT try to fix that residual with FFAccel position offsets — both signs were tested and both are much worse. Adding Output distorts the command trajectory's velocity by $\text{FFAccel} \times \text{jerk}$ (± 1000 counts/s at $J=25000$ with $|\text{FFAccel}|=0.04$), and because the servo's lag depends on command velocity, the "correction" feeds back through the nonlinearity. The structurally correct tool is a **velocity-scheduled lead**: the tap slides along the REAL trajectory, so the servo always executes a legitimate profile point:

```
lead(v) = L0 + slope * |v|    // fit from the LagVsVel table
```

Two scheduling subtleties (both produce lead-in-accel / lag-in-decel residuals if ignored): schedule on the velocity **at the tap**, not the planner's last_vel — the planner runs (CmdDelay - CmdLead) samples in the tap's future — and slew-limit lead changes, since dLead/dt momentarily scales the command velocity. Note User C threads tick every **180 μs = 2 servo samples** — history indexing must account for it. SerialServoLeadSchedule.c implements all of this as a User C thread.

There is additionally a **decel-only lag** (~40 counts) that the constant velocity table cannot see — mostly a fixed braking hysteresis (it barely shrank at half acceleration). The scheduler's DSLOPE term (extra lead while |v| falls) tunes it out; being partly fixed, DSLOPE is calibrated to the profile's acceleration — re-tune (or switch to a constant lead bonus while decelerating) if the machine runs very different accelerations.

Final measured recipe (12 V C018, 4-inch arm, V=2500 A=2500 J=25000): CmdDelay = 2500, schedule lead = $250 + 0.751 \cdot |v| + 0.25 \cdot \text{decel}$, no PID, no FF — tracking within **± 8 counts** open loop (from 80 counts at 1000 counts/s uncompensated). MaxFollowingError can then be tightened to ~50 counts.

Costs to understand:

- The whole machine's physical response shifts later by `CmdDelay` — including feedhold's decel ramp (hard disable/E-stop kill is immediate and unaffected). Jogs inherit the same latency.
- **Change `CmdDelay` only while stationary** — the reference steps by `CmdDelay` × velocity otherwise (the firmware bypasses the taps while the ring refills after a change, so the step happens at the moment of change, not later).
- Soft limits and limit switches still act on the planner (undelayed) stream — they trip *earlier*, which is the safe direction.
- Known limitation: `BACKLASH_LINEAR` direction detection follows the delayed reference, so with `CmdLead > 0` the compensation reaches the output about `CmdLead` late at direction reversals (a brief transient — velocity is near zero at a reversal). Avoid combining mechanical backlash compensation with large `CmdLead` on the same axis, or accept the transient.

Following error

`MaxFollowingError` is checked in both output modes (in `Servo()` for CL, in `SerialServoOutput()` when the input is `SERIAL_SERVO_INPUT_MODE` otherwise). The ~80 ms lag converts to error in proportion to speed (~160 counts at 2000 counts/s) unless feed forward compensates it — budget accordingly, or apply `FFv1` first. A jammed or unplugged servo (position freezes while the command moves) trips the normal following-error kill.

Disabling

Disabling stops sending that servo position commands (after a ~5 ms grace); the servo holds its last position under its own torque, like a stepper holds with current. To go limp, disable torque via the aux mailbox.

Tuning plots (KMotion.exe Step Response screen)

The tuning capture carries servo current / load / supply voltage in the existing gather columns, scaled properly when the plotted axis is a serial servo: *Axis Current* shows a single "Servo Current" trace in Amps, *Supply Voltage* in Volts, and the *Serial Servo Load* plot type shows the servo's own torque estimate in %.

Files

FPGA: `sts_uart.vhd` (byte-stream UART) + integration in `pwms.vhd`. DSP: `DSP_KOGNA\fast.c` (`DoSerialServos` protocol engine, mode handlers), `PC-DSP.h` (modes, `N_SERIAL_SERVOS`), `KMotionDef.h` (registers, globals, aux mailbox), `PC.c` (enable interlock), `Int.c` (ISR hook, gather columns). PC: `KMotion\Graph.*`, `StepDlg.*`, `ConfigDlg.cpp`. Examples: `SerialServoSetID.c`, `SerialServoAxisInit.c`, `SerialServoTuneLag.c`, `SerialServoSweepLag.c`.